

Onyx — The Kits: a developer's guide

Onyx · multi-process OS on Raspberry Pi 4

How an Onyx program is built on the system's shared libraries — the **kits** — with a short example for each. Each kit has its reference, with every operation it exposes (documents 10 to 18, linked in the table below). Companion of the [Developer Guide](#) (the build, the application model, the ports) and of [Kernel Internals](#).

Table of contents

1. One kit per domain
2. Using a kit in a program
3. AppKit — what makes a program run
4. UIKit — the interface
5. SystemKit — talking to the system and the other programs
6. NetKit — the network
7. FileKit — files, folders, archives
8. ImageKit — pictures
9. AudioKit — sound
10. FontKit — fonts and text
11. PrinterKit — printing
12. Adding to a kit, creating a kit
13. Quick reference

1. One kit per domain

An Onyx program does not talk to the kernel, and does not carry copies of common code. It is built on **kits**: shared libraries, one per domain, loaded once for the whole system and bound to each program that uses them.

Kit	Domain	On the card	Sources	Reference
AppKit	What makes a program run: the system's calls, strings, the console, <code>.ini</code> files, starting programs	<code>SD:/lib/appkit.so</code>	<code>user/Kits/appkit</code>	10 — AppKit
UIKit	The interface: windows, widgets,	<code>SD:/lib/uikit.so</code>	<code>user/Kits/uikit</code>	11 — UIKit

Kit	Domain	On the card	Sources	Reference
	dialogs, the theme, icons			
SystemKit	Talking to the system and the other programs: notifications, clipboard, trash, volume, wallpaper, file associations, the dock	<code>SD:/lib/systemkit.so</code>	<code>user/Kits/systemkit</code>	12 — SystemKit
NetKit	The network: HTTP, the FTP volumes	<code>SD:/lib/netkit.so</code>	<code>user/Kits/netkit</code>	13 — NetKit
FileKit	Files and folders, paths, compression, archives	<code>SD:/lib/filekit.so</code>	<code>user/Kits/filekit</code>	14 — FileKit
ImageKit	Pictures: reading, writing, resizing, adjusting	<code>SD:/lib/imagekit.so</code>	<code>user/Kits/imagekit</code>	15 — ImageKit
AudioKit	Sound: playing files, notes, synthesis, the sound output	<code>SD:/lib/audiokit.so</code>	<code>user/Kits/audiokit</code>	16 — AudioKit
FontKit	Fonts: FreeType, the font manager, anti-aliased text	<code>SD:/lib/fontkit.so</code>	<code>user/Kits/fontkit</code>	17 — FontKit
PrinterKit	Printing: the Print dialog, a job's pages	<code>SD:/lib/printerkit.so</code>	<code>user/Kits/printerkit</code>	18 — PrinterKit

Three rules follow from this layout, and they hold for every new development:

- **Reusable code goes into the adequate kit** — not into a new shared header, not copied into an application. If no kit fits the domain, a new kit is created (§12).
- **A program draws on the kits as much as it can**, and never reaches the kernel itself. Only AppKit reads the kernel's table; so the kernel can change without any program being rebuilt.
- **What one program alone uses stays beside that program**, in its own folder.

A kit has **one header**, `<kit>/<kit>.h`, which **declares**; the code is in the library. The memory cost of a kit is paid once: its code is shared by every program that uses it.

2. Using a kit in a program

Including

`user/Kits` is on every include path. A source includes a kit's header by the kit's name, whatever its own folder:

```
#include "appkit/appkit.h"      // AppKit
#include "uikit/uikit.h"        // UIKit
#include "systemkit/systemkit.h" // SystemKit
#include "netkit/netkit.h"       // NetKit
#include "filekit/filekit.h"     // FileKit
#include "imagekit/imagekit.h"   // ImageKit
#include "audiokit/audiokit.h"   // AudioKit
#include "fontkit/uikitface.h"   // FontKit
#include "printerkit/printerkit.h" // PrinterKit
```

Linking

AppKit needs nothing: the kernel loads it and binds it to every program. Every other kit is linked through its **import archive**, a few bytes of stubs and a constructor that opens the library before `main`:

The program is	It links
C++ (an application)	<code>lib/<kit>.imp.a</code>
C (a console tool)	<code>lib/<kit>.imp_c.a</code>

An archive only brings what the program calls: listing a kit a program does not use costs nothing, and the library is not opened. The applications' rules in `user/Makefile` already list UIKit, FileKit, SystemKit and NetKit; a console tool names its kits in `user/BinUtils/Makefile`:

```
# a C tool that uses SystemKit
mytool.elf: KITLIBS = ../lib/systemkit.imp_c.a
mytool.elf: ../lib/systemkit.imp_c.a
```

Versions

A kit's table only grows: a function is never removed nor renamed, so a newer library always runs an older program. A program built against a table that grew needs at least that library: its package says so (`needs = systemkit >= 1.56` in `tools/pkg/packages.ini`), and the package manager installs the library first.

On a PC

The same sources build on a PC for the tests and the screenshots. There is no shared library there: the kits' headers bring their code inline, and UIKit's sources are compiled with the program. Nothing changes in the program's code.

3. AppKit — what makes a program run

`#include "appkit/appkit.h"` — nothing to link.

AppKit is the program's link to the system: files, processes, time, windows, sockets... (the `kapi_*` calls, listed in the Developer Guide), plus the small services every program needs: strings without a C library, the console, a reader of `.ini` files, the starting of other programs.

A console tool: read a file, print to the console.

```
#include "appkit/appkit.h"

int main (void)
{
    char buf[256];
    void *f = kapi_open ("SD:/etc/autostart");
    if (f == 0) { ax_putln ("no such file"); return 1; }
    int n = kapi_read (f, buf, sizeof buf - 1);
    kapi_close (f);
    buf[n > 0 ? n : 0] = '\0';

    ax_putln ("SD:/etc/autostart says:");
    ax_putln (buf);
    return 0;
}
```

Settings from an `.ini` file in the program's own folder:

```
if (app_ini_load ("config.ini") >= 0)
{
    const char *name = app_ini_get ("window", "title", "Untitled");
    int width = app_ini_get_int ("window", "width", 640);
    for (int i = 0; i < app_ini_count (); i++) // or every entry, in the file's order
        ax_putln (app_ini_key (i));
}
```

Strings (no C library needed):

```
char path[96]; int n = 0;
ax_strcat (path, sizeof path, &n, "SD:/docs/"); // never overflows, always terminated
ax_strcat (path, sizeof path, &n, "notes.txt");
char num[12]; ax_itoa (42, num);
if (ax_streq (num, "42")) ax_putln (path);
```

Starting a program — an application by its name, or a file by whatever runs it (a `.bas` program by BASIC, a game by its emulator):

```
lx_launch ("tinypad", "SD:/docs/notes.txt"); // an app, with its arguments
lx_open ("SD:/games/tetris.gb", 0); // a file: by its runner
```

4. UIKit — the interface

`#include "uikit/uikit.h"` — link `lib/uikit.imp.a` (the applications' rules do).

A window is a `Root`; widgets are added to it; `run ()` is the event loop. Everything is in the namespace `uikit`.

A window, a label, a button:

```
#include "appkit/appkit.h"
#include "uikit/uikit.h"
using namespace uikit;
```

```

static Label *g_label;
static int g_count;

static void onClick (Widget &)
{
    char text[32]; int n = 0;
    ax_strcat (text, sizeof text, &n, "clicked ");
    char num[12]; ax_itoa (++g_count, num);
    ax_strcat (text, sizeof text, &n, num);
    g_label->setText (text); // the widget redraws itself
}

int main (void)
{
    Root root (320, 120, "Hello"); // a decorated window
    g_label = new Label (12, 12, 296, 20, "Press the button");
    root.addChild (g_label);
    root.addChild (new Button (12, 44, 100, 28, "Click me", onClick));
    root.run (); // until the window is closed
    return 0;
}

```

Dialogs are modal calls:

```

if (uk_messagebox ("Delete", "Delete this file?", MB_YESNO))
{
    char path[256];
    if (uk_file_open (path, sizeof path, "SD:/docs")) // the file chooser
        g_label->setText (path);
}

unsigned colour = 0x2060C0;
if (uk_color_dialog (&colour, "Pick a colour")) root.setBg (colour);

```

The theme: draw with the palette (C_BG, C_TEXT, C_ACCENT, C_FACE...), never with fixed colours, so that the program follows the user's theme.

An icon — any picture ImageKit reads (BMP today, PNG tomorrow), given as icons are drawn (magenta = see-through):

```

#include "uikit/bmp.h"

int w, h;
unsigned *px = ui::icon_load ("SD:/apps/tinypad.app/icon.bmp", &w, &h);
if (px) { /* draw it ... */ delete [] px; }

```

A timer or polling: derive from `Root` and override `onTick ()` (called about 60 times a second).

5. SystemKit — talking to the system and the other programs

`#include "systemkit/systemkit.h"` — link `lib/systemkit.imp.a` (C++) or `lib/systemkit.imp_c.a` (C). The one header brings every subject below (a C program gets those written in C: notifications, volume, preload list, the applets' protocol).

A notification (a bubble under the menu bar; `notify_action` adds what a click starts):

```
#include "systemkit/systemkit.h"
```

```
notify ("Backup", "3 files copied");  
notify_action ("Updates", "2 updates available", "control pkgman");
```

The clipboard, shared by every application:

```
#include "systemkit/systemkit.h"
```

```
clip_set_text ("copied text");
```

```
char buf[1024];  
if (clip_get_text (buf, sizeof buf)) { /* paste buf */ }  
  
int w, h;  
unsigned *px = clip_get_image (&w, &h); // a picture, if one was copied  
if (px) { /* ... */ delete [] px; }
```

The trash — a file is moved there instead of being deleted:

```
#include "systemkit/systemkit.h"
```

```
if (trash_move ("SD:/docs/old.txt")) notify ("Trash", "old.txt moved to the trash");  
int n = trash_count (); // what the trash holds
```

Opening a file with the right application (the user's file associations):

```
#include "systemkit/systemkit.h"
```

```
char app[32];  
if (fa_app_for ("SD:/photos/cat.png", app, sizeof app)) { /* "imageview" */ }  
fa_open ("SD:/photos/cat.png"); // a folder, an app, a document: opened
```

The volume, the wallpaper, the dock — a settings program changes the file and tells the system:

```
#include "systemkit/systemkit.h"
```

```
volume_save (7, 0); // 0..10, not muted: kept for the next  
start
```

```
Wallpaper wp;  
wp_load (wp); // SD:/etc/wallpaper.ini  
/* change wp ... */  
wp_save (wp);
```

```
dock_reload (); // the dock reads its settings again
```

Its part (a header of its own, beside systemkit.h)	Subject
notify.h	Notifications
clipboard.h (clipproto.h: its protocol)	The clipboard
trash.h	The trash
fileassoc.h	Which application opens which file
volume.h	The master volume, the output, each

Its part (a header of its own, beside <code>systemkit.h</code>)	Subject
	program's volume
<code>wallpaper.h</code>	The wallpaper's settings and its painter
<code>dockconf.h</code>	The dock's settings
<code>preloadini.h</code>	The programs loaded ahead at boot
<code>applet_proto.h</code>	A settings applet shown inside the Control Panel

6. NetKit — the network

`#include "netkit/netkit.h"` — link `lib/netkit.imp.a` (C++) or `lib/netkit.imp_c.a` (C). The `HttpClient` class is apart: `#include "netkit/http.hpp"`.

A page fetched (HTTP/1.0, no allocation: the caller gives the buffer):

```
#include "appkit/appkit.h"
#include "netkit/netkit.h"

static char buf[32 * 1024];

int main (void)
{
    http_response r;
    if (http_get ("http://example.com/", buf, sizeof buf, &r) < 0 || !r.ok)
    {
        ax_putln ("request failed");
        return 1;
    }
    kapi_stdout_write (r.body, (unsigned) r.body_len);
    return 0;
}
```

A web API (HTTP/1.1, headers, HTTPS with the TLS transport) — the `HttpClient` class:

```
#include "netkit/http.hpp"

static char buf[64 * 1024];
HttpClient http;
http.user_agent ("MyApp/1.0").accept ("application/json");
HttpResponse r = http.get ("http://api.example.com/status", buf, sizeof buf);
if (!r.is_error ()) { /* r.status, the body in buf */ }
```

An FTP server as a volume: once logged in, `FTP:host/dir/file` is a path like any other, for every program.

```
#include "netkit/netkit.h"

if (ftpfs_login ("ftp.example.com", "me", "secret", 1)) // 1: remembered
{
    void *f = kapi_open ("FTP:ftp.example.com/pub/readme.txt");
    /* kapi_read ... kapi_close */
}
```

The raw sockets (`kapi_tcp_connect`, `kapi_tcp_send`...) are AppKit's; NetKit is where protocols built on them belong.

7. FileKit — files, folders, archives

```
#include "filekit/filekit.h" — link lib/filekit.imp.a.
```

A whole file, read and written:

```
#include "filekit/filekit.h"

void *data; unsigned n;
if (fk_load ("SD:/docs/notes.txt", &data, &n) == 0)
{
    /* data: n bytes, a 0 after them */
    fk_save ("SD:/docs/notes.bak", data, n);
    fk_free (data);
}
```

Folders and paths:

```
#include "filekit/filekit.h"

char path[FS_PATHL];
fs_join (path, sizeof path, "SD:/docs", "report.txt");
if (!fs_exists (path)) { /* ... */ }

fk_mkdirs ("SD:/backup/2026/docs");           // every folder of the path
fk_copy ("SD:/docs", "SD:/backup/2026/docs", 0, 0); // a file, or a folder and all it holds
fk_remove ("SD:/tmp/work");                   // the same, removed
```

A ZIP archive, read:

```
char err[80];
fk_zip *z = fk_zip_open ("SD:/downloads/pack.zip", err, sizeof err);
if (z)
{
    struct fk_zip_entry e;
    for (int i = 0; i < fk_zip_count (z); i++)
        if (fk_zip_entry (z, i, &e)) ax_putln (e.name);
    fk_zip_extract_all (z, 0, "SD:/downloads/pack", 0, 0);
    fk_zip_close (z);
}
```

A ZIP archive, written:

```
fk_zipw *w = fk_zipw_create ("SD:/backup/docs.zip");
fk_zipw_add (w, "SD:/docs", 0); // the folder and all it holds
fk_zipw_close (w, 0, 0, err, sizeof err);
```

Other archive formats (tar, tar.gz, gzip) are read through the same calls (`fk_arc_*`); a program asks the library which formats it handles (`fk_arc_formats`) instead of keeping its own list.

8. ImageKit — pictures

```
#include "imagekit/imagekit.h" — link lib/imagekit.imp.a.
```

ImageKit is the system's one picture codec: BMP, GIF, PNG, JPEG, PCX and WebP read; PNG, JPEG, BMP and GIF written.

A picture read, turned, saved in another format:

```
#include "imagekit/imagekit.h"

ik_image *im = ik_load ("SD:/photos/cat.jpg", 0);
if (im)
{
    int w = ik_width (im), h = ik_height (im);
    unsigned *px = ik_pixels (im);           // w * h pixels, 0xAARRGGBB
    ik_rotate (im, 1);                       // a quarter turn clockwise
    ik_save (im, "SD:/photos/cat.png", 90);   // the format: the extension's
    ik_image_free (im);
}
```

Pixels of the program's own, saved:

```
ik_image *shot = ik_image_from (pixels, width, height, width);
ik_save (shot, "SD:/screenshots/shot.png", 0);
ik_image_free (shot);
```

The formats, asked — a file chooser's filter follows what the library knows:

```
struct ik_format f[16];
int n = ik_formats (f, 16);
```

Resizing (`ik_scale`), flipping (`ik_flip`), the photo adjustments and the EXIF orientation are in the same header.

9. AudioKit — sound

```
#include "audiokit/audiokit.h" — link lib/audiokit.imp.a.
```

Several programs play at once: the system mixes them, each with its own volume (Control Panel ► Sound).

A sound file (WAV, MP3, OGG, FLAC, MIDI, FM Tracker songs), played in the background:

```
#include "audiokit/audiokit.h"

if (ak_play ("SD:/music/song.mp3", 0) < 0) ax_putln (ak_play_error ());
ak_play_volume (80);                               // 0..100
/* ... the program goes on ... */
ak_play_stop ();
```

Notes (the General MIDI instruments, a SoundFont):

```
ak_note_on (0, 60, 100);                           // channel, key (60 = middle C),
velocity
kapi_msleep (400);
ak_note_off (0, 60);
```

A sound effect (the FM synthesizer's voices — what the games use):

```
ak_fm_start (0, 440000, 0, 200);                     // voice 0, 440 Hz (in milli-Hz),
volume 0..255
```

```
kapi_msleep (120);
ak_fm_stop (0);
```

The program's own samples (an emulator, a synthesizer): stereo, 16 bits, 44.1 kHz (AUDIOKIT_RATE).

```
if (ak_out_open (256, 1024) == 1)
{
    short frames[256 * 2];
    /* fill frames ... */
    ak_out_write (frames, 256);           // waits for room
    ak_out_close ();
}
```

10. FontKit — fonts and text

#include "fontkit/uikitface.h" or "fontkit/fonts.h" — link lib/fontkit.imp.a.

FontKit is FreeType for the applications, with the card's fonts (SD:/res/fonts).

Anti-aliased text in a UIKit application — one call before the widgets are built:

```
#include "uikit/uikit.h"
#include "fontkit/uikitface.h"
using namespace uikit;

int main (void)
{
    ft_uikit_install ("DejaVu Sans", 13);           // every widget's text from now on
    Root root (400, 200, "Smooth text");
    root.addChild (new Label (12, 12, 376, 24, "FreeType draws this"));
    root.run ();
    return 0;
}
```

The font manager — the families installed, a font at a size, text measured (a word processor, a canvas):

```
#include "fontkit/fonts.h"

fnt::init ();
for (int i = 0; i < fnt::count (); i++) ax_putln (fnt::name (i));

int fam = fnt::find ("DejaVu Serif");
fnt::Font *f = fnt::get (fam, 0, 14 * 64);           // style 0 (regular), 14 px in 26.6
int width = fnt::advance (f, 'A');                 // in 26.6 units
```

11. PrinterKit — printing

#include "printerkit/printerkit.h" — link lib/printerkit.imp.a.

A program prints in four steps: the Print dialog, a job, its pages, the end. Lengths are points (1/72 inch) from the page's top-left corner; colours are 0xRRGGBB. The pages are recorded: the print service turns them into what the printer takes — or into a PDF file.

```
#include "printerkit/printerkit.h"

PrintSetup s;
print_setup_default (&s);
PrintDialogInfo di = { sizeof di, "My document", 1, 0, 0, 0, 0 };
```

```

if (print_dialog (&s, &di)) // the printer, the paper, the pages
{
    PrintJob *j = print_begin (&s, "My document");
    int font = print_font (j, "DejaVu Sans", 0);
    if (print_page (j, 0, 0)) // 0, 0: the paper chosen
    {
        print_text (j, font, 12, 72, 84, "Hello, printer", 0x000000);
        print_line (j, 72, 90, s.paper_w - 72, 90, 0.75f, 0x808080);
    }
    print_end (j); // to the print queue
}

```

Rectangles, paths (lines and curves, filled or stroked) and pictures are drawn the same way (`print_rect`, `print_path_*`, `print_image`).

12. Adding to a kit, creating a kit

Before writing a helper, look for it in the kits. If two programs could use it, it belongs to a kit.

Adding a function to a kit

1. **Declare** it in the kit's header, with a comment that says what it does.
2. **Write its code** in the kit's source (for SystemKit, NetKit and `filekit/fsutil.h`: the `.inc` beside the header).
3. **Build**: the kit's table (`<kit>.abi`) gets the new name at its end. Commit the `.abi`. A name is never removed nor renamed: a function that is no longer needed keeps a body.
4. **Raise the package's version** (`tools/pkg/versions.ini`: `1.<the table's size>.0`) and what the applications need (`needs = <kit> >= 1.<size>` in `tools/pkg/packages.ini`).
5. **Document** it: its comment in the header is its documentation — run `python tools/docgen/kitdocs.py` (the kit's reference follows), add an example here if it deserves one, then `python docs/build_docs.py`.

Creating a kit

When a domain has no kit (SystemKit and NetKit are the models — small, with no dependency):

- a folder `user/Kits/<kit>/:` **one header a program includes**, `<kit>/<kit>.h` — it may bring a header per subject, each declaring its functions (`XK_API int f (...);`) with a `.inc` beside it for the code — and one source `<kit>.cpp` that compiles the code into the library;
- its rules in `user/Makefile` (copy SystemKit's block): the library `lib/<kit>.so`, its import archives `lib/<kit>.imp.a` (C++) and `lib/<kit>.imp_c.a` (C);
- its package in `tools/pkg/packages.ini` (`required = 1`, `files = lib/<kit>.so`).

The header of a kit a C program may use is written in C.

13. Quick reference

I want to...	Kit	Call
read or write a file	AppKit	<code>kapi_open</code> , <code>kapi_read</code> , <code>kapi_save_file</code>
load a whole file in memory	FileKit	<code>fk_load</code> , <code>fk_save</code>

I want to...	Kit	Call
copy or remove a folder	FileKit	fk_copy, fk_remove
build a path	FileKit	fs_join, fk_path_join
read or write a ZIP	FileKit	fk_zip_open, fk_zipw_create
print to the console	AppKit	ax_puts, ax_putln
read an <code>.ini</code> file	AppKit	app_ini_load, app_ini_get
start another program	AppKit	lx_launch, lx_open
open a window with widgets	UIKit	Root, Label, Button...
ask the user (message, file, colour)	UIKit	uk_messagebox, uk_file_open, uk_color_dialog
load an icon	UIKit	ui::icon_load
show a notification	SystemKit	notify, notify_action
copy / paste	SystemKit	clip_set_text, clip_get_text
move a file to the trash	SystemKit	trash_move
open a file with its application	SystemKit	fa_open
fetch a web page	NetKit	http_get, HttpClient
reach an FTP server	NetKit	ftpfs_login, then FTP: paths
load or save a picture	ImageKit	ik_load, ik_save
resize or turn a picture	ImageKit	ik_scale, ik_rotate
play a sound file	AudioKit	ak_play
play a note or an effect	AudioKit	ak_note_on, ak_fm_start
send samples to the output	AudioKit	ak_out_open, ak_out_write
draw smooth text	FontKit	ft_uikit_install, fnt::get
print	PrinterKit	print_dialog, print_begin, print_page, print_end